



INTEGRATED REFERENCE

AI·Python 핵심정리

Python과 데이터 처리부터 머신러닝, 신경망, 자연어 처리,
Transformer, LLM 평가·안전까지의 핵심 개념과 수식·코드 패턴을
한 문서로 정리한 정보형 레퍼런스.



구성

01 Python · API · JSON

자료형, 컬렉션, 흐름 제어, 함수, HTTP, requests, JSON, LLM API

02 NumPy · Pandas

ndarray, shape, axis, broadcasting, 필터링, 집계, pivot, 파일 입출력

03 시각화 · EDA

차트 선택, Matplotlib, Seaborn, 결측치, 이상치, 구간화, Titanic 패턴

04 ML 기초 · 검증

학습 정의, 회귀·분류, 지표, 과적합, validation, K-fold, 군집화

05 회귀 · 신경망

최소제곱, 최대우도, 로지스틱 회귀, ReLU, 최적화, 역전파, PyTorch

06 NLP · Transformer

토큰, embedding, RNN·LSTM·GRU, Seq2Seq, attention, BERT, T5

07 LLM · 평가 · 안전

사전학습, scaling, SFT·RLHF, decoding, prompting, RAG, 평가, 한계

08 CNN · 이미지 모델

합성곱, 수용영역, pooling, AlexNet, VGG, ResNet, MobileNet

09 ViT · 학습 전략

이미지 attention, positional encoding, 활성화, 초기화, 정규화, 전이학습

Appendix

핵심 수식, 혼동하기 쉬운 비교, 코드 흐름 요약

Python · API · JSON

Python 실행 흐름과 자료구조의 동작, 외부 API 요청과 중첩 JSON 응답 처리에 필요한 핵심 규칙.

변수와 기본 자료형

할당과 이름

- `name = value` 는 값을 객체에 저장한 뒤 이름이 그 객체를 참조하게 한다.
- 이름은 문자 또는 밑줄로 시작하며 숫자로 시작할 수 없다.
- `class`, `for`, `return` 같은 예약어는 변수 이름으로 사용할 수 없다.

대표 자료형

- `int`: 정수, `float`: 부동소수점 실수
- `str`: 따옴표로 감싼 문자열
- `bool`: `True` 또는 `False`
- `None`: 값이 없음을 나타내는 단일 객체

표현	결과	의미
<code>17 / 5</code>	<code>3.4</code>	일반 나눗셈은 실수 결과
<code>17 // 5</code>	<code>3</code>	몫을 내림한 floor division
<code>17 % 5</code>	<code>2</code>	나머지
<code>2 ** 5</code>	<code>32</code>	거듭제곱
<code>int(3.9)</code>	<code>3</code>	0 방향으로 소수부 제거

형변환은 원본 변수를 자동 변경하지 않는다. `int(text)` 는 새 정수 객체를 만들며 `text` 자체는 계속 문자열이다. 자료형은 `type(value)` 로 확인한다.

문자열·리스트·딕셔너리

인덱스와 슬라이스

`a[start:stop:step]` 에서 `stop` 은 포함하지 않는다. 음수 `index` 는 뒤에서부터 세며 `a[::-1]` 은 역순 복사다.

변경 메서드

`append`, `insert`, `remove`, `pop`, `sort` 는 list 상태를 바꾼다. `append` 와 `sort` 의 반환값은 `None` 이다.

새 값을 만드는 함수

`sorted(values)` 는 정렬된 새 list를 반환하고 원본을 그대로 둔다. `str` 도 불변이라 연산 결과가 새 문자열이다.

```

values = [3, 1, 2]
result = values.sort()
# result == None
# values == [1, 2, 3]

copy = sorted(values, reverse=True)
# copy == [3, 2, 1], values는 그대로

```

딕셔너리 연산	동작
<code>d[key]</code>	키의 값을 반환. 키가 없으면 <code>KeyError</code>
<code>d.get(key, default)</code>	키가 없으면 <code>default</code> 를 반환하며 새 키를 추가하지 않음
<code>key in d</code>	키 존재 여부
<code>d.items()</code>	<code>(key, value)</code> 쌍을 순회
<code>d.pop(key)</code>	항목을 제거하고 값을 반환

조건과 반복

`0`, 빈 문자열, 빈 `list`/`dict`/`set`, `None` 은 조건식에서 `Falsy`다. 나머지 대부분의 객체는 `Truthy`다. `and` 와 `or` 는 `Boolean`으로 강제 변환하지 않고 선택된 피연산자 자체를 반환할 수 있다.

반복 범위

`range(start, stop, step)` 역시 `stop`을 포함하지 않는다. `continue` 는 현재 회차의 남은 코드를 건너뛰고, `break` 는 반복을 즉시 끝낸다.

for-else

반복이 `break` 없이 정상 종료되면 `else` 가 실행된다. 검색 성공 시 `break`, 끝까지 실패한 경우 `else`를 사용하는 패턴이 대표적이다.

`enumerate(iterable)` 는 원소와 `index`를 함께 순회한다. 기본 `index`는 0부터 시작하며 `enumerate(items, start=1)` 처럼 시작 번호를 지정할 수 있다.

```

for item in items:
    if not is_valid(item):
        continue
    if matches(item):
        found = item
        break
else:
    found = None

```

함수와 실행 범위

- `def` 는 함수를 정의하고 호출 시 새 지역 `scope`가 만들어진다.
- `return` 은 값을 호출 위치로 전달하고 그 함수 실행을 종료한다. 명시하지 않으면 `None` 을 반환한다.
- `print` 는 화면 출력이라는 부수효과이며 함수 결과 전달과는 별개다.
- 지역 변수는 기본적으로 함수 밖에서 직접 접근할 수 없다. 전역 상태 변경은 결합도를 높이므로 반환값 중심 구성이 명확하다.

- `*args` 는 여러 위치 인자를 tuple로, `**kwargs` 는 여러 키워드 인자를 dict로 묶는다.

```
def average(values):
    if not values:
        return None
    return sum(values) / len(values)
```

HTTP와 requests

메서드	일반적 의미	메서드	일반적 의미
GET	자원 조회	POST	새 자원 생성 또는 처리 요청
PUT	자원 전체 교체	PATCH	자원 일부 수정
DELETE	자원 삭제	OPTIONS	지원 기능 확인

상태 범위	의미	대표값
2xx	요청 성공	200 OK, 201 Created
4xx	클라이언트 요청 문제	400 Bad Request, 401 Unauthorized, 404 Not Found
5xx	서버 처리 문제	500 Internal Server Error

```
import requests
import json

try:
    response = requests.get(url, timeout=5)
    response.raise_for_status()
    data = response.json()
except requests.exceptions.Timeout:
    ...
except requests.exceptions.ConnectionError:
    ...
except requests.exceptions.HTTPError:
    ...
except json.JSONDecodeError:
    ...
```

status_code 만으로는 충분하지 않다. 연결 실패에는 응답 객체가 없을 수 있고, 성공 상태여도 본문 구조가 예상과 다르거나 JSON 파싱이 실패할 수 있다. 제한 시간, HTTP 오류, 파싱 오류를 각각 다룬다.

Response 요소	내용
<code>status_code</code>	HTTP 상태 코드
<code>text</code>	응답 본문 문자열
<code>json()</code>	JSON 본문을 Python 객체로 변환하는 메서드
<code>headers</code>	응답 헤더 mapping
<code>elapsed</code>	요청부터 응답까지 경과 시간
<code>url</code>	최종 요청 URL

JSON 중첩 구조와 LLM API

JSON의 array는 Python `list`, object는 `dict`, string-number-boolean-null은 대응하는 기본 자료형으로 변환된다. 바깥 구조부터 한 단계씩 확인한다.

```
data = [{"name": "Kim", "address": {"city": "Seoul"}}]
city = data[0]["address"]["city"]
```

```
# LLM 응답의 대표 접근 형태
text = data["choices"][0]["message"]["content"]
tokens = data["usage"]["total_tokens"]
```

요청 본문

- `model`: 사용할 모델
- `messages`: role과 content의 배열
- `temperature`: 표본 분포의 다양성 조절
- `max_tokens`: 생성 길이 상한

운영 경계

- `choices`가 비었는지 먼저 확인
- 응답 구조와 finish reason 확인
- token 사용량과 오류 응답 기록
- 생성 코드 사실은 별도 검증

API 키는 코드·노트북·로그에 직접 남기지 않는다. 환경 변수에서 읽고 `.env`는 버전 관리에서 제외한다. 노출된 키는 즉시 폐기하고 재발급한다.

NumPy · Pandas

배열 기반 벡터 연산, shape와 axis, 표 형식 데이터의 선택·집계·재구성.

ndarray와 벡터화

생성	의미	예시 결과
<code>np.array([1, 2])</code>	list에서 ndarray 생성	shape (2,)
<code>np.arange(1, 7, 2)</code>	stop 미포함 등간격	[1, 3, 5]
<code>np.linspace(0, 1, 5)</code>	양끝 포함, 원소 수 지정	같은 간격 5개
<code>np.zeros((2, 3))</code>	0으로 채운 배열	shape (2, 3)
<code>np.ones((2, 3))</code>	1로 채운 배열	shape (2, 3)

Python list

[1, 2] + [3] 은 연결, [1, 2] * 2 는 반복이다.
원소별 수치 계산이 아니다.

NumPy array

`arr + 2`, `arr * 2` 는 각 원소에 벡터화되어 적용된다.
일반적으로 한 배열은 동일 dtype을 갖는다.

$$\text{shape} = \text{각 축의 길이} \cdot \text{ndim} = \text{축의 수} \cdot \text{size} = \text{전체 원소 수}$$

index, reshape, transpose

```
arr = np.arange(12).reshape(3, 4)
arr[1, 2]      # 2번째 행, 3번째 열
arr[:, 1]      # 모든 행의 2번째 열
arr.T          # shape (4, 3)
arr.reshape(2, -1) # 전체 원소 수에 맞춰 두 번째 차원 추론
```

`reshape` 의 전체 원소 수는 변하지 않는다. `-1` 은 한 축에서만 사용할 수 있으며 나머지 차원으로 자동 계산된다.

axis와 집계

2차원 배열 기준	줄어드는 축	남는 결과
<code>sum(axis=0)</code>	행 방향을 접어 열별 집계	열 개수 길이의 1차원 배열
<code>sum(axis=1)</code>	열 방향을 접어 행별 집계	행 개수 길이의 1차원 배열
<code>sum()</code>	모든 축	scalar

`mean`, `sum`, `max`, `min`, `std` 도 같은 axis 규칙을 따른다.

broadcasting과 Boolean indexing

broadcasting은 두 shape를 뒤쪽 차원부터 비교한다. 각 차원이 같거나 한쪽이 1이면 확장 가능하다. 예를 들어 `(2, 3)` 배열과 `(3,)` 벡터의 합은 벡터를 각 행에 적용해 `(2, 3)` 을 만든다.

```
arr = np.array([1, 2, 3, 4, 5])
selected = arr[(arr > 2) & (arr < 5)] # [3, 4]

# 배열 조건은 and/or가 아니라 &|
# 각 비교식은 괄호로 분리
```

정렬 함수	반환 내용
<code>np.sort(arr)</code>	오름차순으로 정렬된 값
<code>np.sort(arr)[::-1]</code>	내림차순 값
<code>np.argsort(arr)</code>	정렬 순서를 만드는 원래 index

Series와 DataFrame

Series

하나의 값 배열과 index로 구성된 1차원 구조.
`df['sales']` 는 Series를 반환한다.

DataFrame

행 index와 열 columns를 가진 2차원 표.
`df[['sales']]` 는 열 하나여도 DataFrame이다.

확인 메서드	내용
<code>head()</code> , <code>tail()</code>	앞·뒤 행 일부
<code>shape</code> , <code>dtypes</code>	행·열 수, 열별 자료형
<code>info()</code>	열, non-null 개수, dtype, 메모리 요약
<code>describe()</code>	수치형 열의 개수·평균·표준편차·분위수
<code>index</code> , <code>columns</code> , <code>values</code>	축 label과 내부 값

행·열 선택

```
# Boolean 행 필터
adults = df[df['age'] >= 30]
target = df[(df['age'] >= 30) & (df['city'] == 'Seoul')]

# label 기반
df.loc[df['sex'] == 'female', ['age', 'fare']]

# 정수 위치 기반
df.iloc[:5, [0, 3]]
```

.loc 는 label과 Boolean 조건, .iloc 는 0부터 시작하는 정수 위치를 사용한다. 정수형 index가 있을 때도 두 기준은 별개다.

isin 은 여러 후보 포함 여부, str.contains 는 문자열 포함, str.startswith 는 접두어 조건을 만든다.
nlargest · nsmallest 는 상·하위 행, sample(random_state=...) 은 재현 가능한 무작위 표본을 선택한다.

groupby·agg·pivot

```
# 한 기준, 한 집계
df.groupby('department')['salary'].mean()

# 여러 집계와 일반 열 복원
summary = (df.groupby(['department', 'year'])['sales']
            .agg(['sum', 'mean', 'count'])
            .reset_index())

# 행·열 축으로 재구성
table = df.pivot_table(
    index='department', columns='year',
    values='sales', aggfunc='sum'
)
```

집계	결측치 처리	의미
count()	NaN 제외	열별 유효값 수
size()	NaN 포함	그룹 전체 행 수
mean()	NaN 제외	유효값 평균
sum()	기본적으로 NaN 제외	그룹 합계

파일 입출력과 재현성

- pd.read_csv(path, encoding='utf-8-sig') 또는 환경에 따라 cp949 인코딩을 사용한다.
- df.to_csv(path, index=False) 는 DataFrame index가 불필요한 파일 열로 기록되는 것을 막는다.
- np.random.seed(42), random_state=42 처럼 난수 조건을 고정하면 같은 실험을 재현할 수 있다.
- 파생열은 df['sales'] = df['price'] * df['quantity'] 처럼 같은 행의 열 연산으로 만든다.

시각화 · EDA

변수의 종류와 분석 목적에 맞는 차트, 데이터 품질 확인과 전처리, 그룹별 탐색 분석.

목적별 차트 선택

질문	대표 차트	주요 해석
범주별 크기는 어떻게 다른가	bar plot	공통 기준선에서 범주 크기 비교
시간에 따라 어떻게 변하는가	line plot	추세·변곡·계절성
두 수치형 변수는 어떤 관계인가	scatter plot	방향·형태·군집·이상치
한 수치형 변수는 어떻게 분포하는가	histogram	중심·퍼짐·왜도·다봉성
범주별 분포와 이상치는 어떤가	boxplot	중앙값·사분위 범위·이상치 후보
수치형 열의 선형 상관은 어떤가	correlation heatmap	상관계수의 부호와 크기

차트는 관계를 보여 주지만 인과를 증명하지 않는다. 상관계수는 선형 관계에 민감하며 이상치와 집단 혼합의 영향을 받는다.

Matplotlib·Seaborn 구성

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.rcParams['font.family'] = 'Malgun Gothic'
plt.rcParams['axes.unicode_minus'] = False

plt.figure(figsize=(8, 4))
plt.bar(category, sales, label='매출')
plt.title('범주별 매출')
plt.xlabel('범주'); plt.ylabel('매출')
plt.xticks(rotation=45)
plt.legend()
plt.tight_layout()
plt.show()
```

- 한글 네모 표시는 한글 glyph를 포함한 font 지정으로 해결한다.
- `axes.unicode_minus=False` 는 일부 한글 font에서 음수 부호가 깨지는 현상을 줄인다.
- `rotation` 은 긴 범주명 겹침, `tight_layout` 은 그림 영역 잘림을 줄인다.
- Seaborn `barplot` 은 원자료에서 범주별 평균과 불확실성을 기본 집계할 수 있으므로 단순 합계 막대와 구분한다.

```
corr = df.corr(numeric_only=True)
sns.heatmap(corr, annot=True, cmap='coolwarm', vmin=-1, vmax=1)
```

결측치와 이상치

확인·처리	핵심 성질
<code>df.isnull().sum()</code>	열별 결측치 수
<code>fillna(mean)</code>	대칭적 분포의 수치형 열에서 고려
<code>fillna(median)</code>	치우침·이상치에 평균보다 강건
<code>fillna(mode()[0])</code>	범주형 열의 최빈값 사용
<code>dropna()</code>	간단하지만 표본 손실과 편향 가능성
<code>quantile(0.9)</code>	상위 10% 경계 같은 분위수 계산
<code>describe()</code> , <code>boxplot</code>	범위·사분위수·극단값 확인

결측값은 완전히 무작위가 아닐 수 있다. 결측 여부 자체가 결과와 관련된다면 단순 삭제나 일괄 평균 대체가 분포를 왜곡할 수 있다. 처리 전후의 행 수, 분포, 그룹별 비율을 함께 비교한다.

category dtype의 새 값: 등록되지 않은 값을 넣기 전에 `series.cat.add_categories('No Data')` 로 범주를 추가한 뒤 `fillna` 한다.

구간화·파생 변수·변환

pd.cut

값의 범위를 경계 또는 같은 폭으로 나눈다. 분포가 치우치면 각 구간의 관측치 수가 크게 다를 수 있다.

pd.qcut

분위수 기준으로 나누어 각 구간의 관측치 수를 비슷하게 만든다. 동일값이 많으면 경계 중복을 처리해야 한다.

```
work = original.copy()
work['family'] = work['sibsp'] + work['parch']
work['fare_band'] = pd.qcut(work['fare'], 5)
work['age_band'] = pd.cut(work['age'], 10)
work['gender'] = work['sex'].apply(lambda x: 1 if x == 'male' else 0)
```

`apply` 는 행·열 값에 사용자 함수를 적용하지만 단순 산술·문자열 연산은 벡터화된 내장 연산이 더 명확하고 빠를 수 있다.

그룹별 EDA 구조

```
# 비율과 규모를 함께 확인
by_group = (df.groupby(['sex', 'pclass'])['survived']
            .agg(['sum', 'mean', 'count'])
            .reset_index()
            .sort_values('mean', ascending=False))

# 두 축 비교
matrix = df.pivot_table(
    index='sex', columns='pclass', values='survived',
    aggfunc=['sum', 'mean']
)
```

- 이진 변수의 평균은 1의 비율이다. `survived.mean()` 은 생존율, `sum()` 은 생존 인원이다.
- 전체 평균은 성별·등급·연령 등 하위 집단 차이를 숨길 수 있으므로 비율과 표본 수를 함께 본다.
- 상위 운임 집단은 `fare >= fare.quantile(.9)` 처럼 선택할 수 있다.
- `deck`의 결측 여부별 결과 차이는 `deck.notnull()` 을 그룹 변수로 사용해 확인할 수 있다.

전처리 안전성

1. 원본을 보존하고 작업본을 만든다.
2. `shape`, `dtype`, 결측치, 중복, 분포 범위를 기록한다.
3. 대체·삭제·범주 변환의 기준을 열별로 정한다.
4. 각 단계 후 행 수와 요약통계를 전 단계와 비교한다.
5. 모델링이 포함되면 평균·스케일·범주 사전은 `train`에서만 학습해 `validation/test`에 적용한다.

ML 기초 · 검증

데이터에서 함수를 선택하는 학습의 구조, 일반화 성능을 추정하는 평가 설계, 지도·비지도학습의 대표 방법.

AI · ML · DL과 학습 구성요소

AI ⊃ Machine Learning ⊃ Deep Learning

AI에는 규칙·탐색·최적화 기반 시스템도 포함된다. ML은 명시적으로 모든 규칙을 작성하는 대신 데이터로 함수 또는 규칙을 추정하고, DL은 다층 신경망을 사용하는 ML의 하위 분야다.

요소	정의
sample / observation	데이터셋의 한 사례
feature, X	예측에 사용되는 수치·범주·텍스트·이미지·음성 등의 입력
label / target, Y	예측하려는 정답
model, f	입력에서 출력을 계산하는 후보 함수
parameter	데이터로 학습되는 weight-coefficient
hyperparameter	학습 전 정하는 learning rate, 깊이, K 등
loss	학습 중 최소화하는 오차 목적
metric	모델의 유용성을 해석하는 평가값

$$Y = f^*(X) + \varepsilon$$
$$X \in \mathbb{R}^p, f^*: \mathbb{R}^p \rightarrow \mathbb{R}, \varepsilon \perp X, E[\varepsilon] = 0$$

f^* 는 관측할 수 없는 평균 관계, ε 는 측정오차·누락 변수·무작위성을 포함한 잡음

$$\hat{f} = \arg \min_{f \in F} (1/n) \sum_{i=1}^n L(y_i, f(x_i))$$

가설공간 F 안에서 평균 손실을 가장 작게 하는 후보를 선택

지도학습과 비지도학습

유형	입력	산출물	대표 과제
회귀	X와 연속값 Y	수치 예측	가격·매출·온도
분류	X와 범주 Y	class 또는 확률	스팸·질병·이미지 종류
비지도학습	label 없는 X	구조·요약·표현	군집화·PCA·밀도·이상치

분류의 decision boundary는 feature 공간에서 class 영역을 나누는 경계다. 예측 관계가 강해도 원인-결과 관계를 증명하지는 않으며, 예측과 인과 추론은 구분된다.

회귀 지표

MSE = (1/n) \sum (y_i - \hat{y}_i)^2 \cdot RMSE = \sqrt{MSE}

- MSE는 큰 오류를 제곱으로 강하게 반영하며 단위가 목표값 단위의 제곱이다.
- RMSE는 목표값과 같은 단위로 되돌아가 전형적인 오차 규모를 해석하기 쉽다.
- 둘 다 이상치에 민감하다.

R^2 = 1 - \sum (y_i - \hat{y}_i)^2 / \sum (y_i - \bar{y})^2

R²는 평균만 예측하는 기준과 비교한 상대 설명력이다. test에서 음수가 될 수 있으며 이 경우 해당 데이터에서 평균 예측보다 제곱오차가 크다. 절대 오차 규모는 RMSE와 함께 본다.

분류 지표와 오류 유형

지표	식	해석
Accuracy	(TP+TN)/(TP+FP+FN+TN)	전체 중 정답 비율
Precision	TP/(TP+FP)	양성 예측 중 실제 양성
Recall / TPR	TP/(TP+FN)	실제 양성 중 탐지
Specificity / TNR	TN/(TN+FP)	실제 음성 중 음성 판정
NPV	TN/(TN+FN)	음성 예측 중 실제 음성
F1	2·Precision·Recall/(Precision+Recall)	Precision과 Recall의 조화평균

FP는 Type I error, FN은 Type II error로도 부른다. ROC는 threshold를 바꾸며 TPR 대 FPR을 그린 곡선이고 AUC는 그 아래 면적이다. 클래스 불균형에서는 accuracy만으로 소수 class 탐지력을 판단할 수 없다.

과소적합·과적합·일반화

과소적합

가설공간 또는 학습이 지나치게 단순해 train과 새 데이터 모두 오차가 큰 상태. bias가 큰 편이다.

과적합

train의 우연한 변동까지 맞춰 train 오차는 작지만 새 데이터 성능이 불안정한 상태. variance가 큰 편이다.

대표성 있는 데이터, 적절한 모델 복잡도, 정규화, validation·교차검증을 사용해 일반화 성능을 관리한다. test를 반복 확인하며 모델을 바꾸면 test에도 과적합한다.

train · validation · test

1. **train**: 파라미터 학습
2. **validation** 또는 **CV**: 모델 구조·hyperparameter·threshold 선택
3. **test**: 모든 선택이 끝난 뒤 최종 일반화 성능 확인

데이터 누수: 전처리 통계, feature 선택, 모델 선택에 validation/test 정보를 미리 사용하면 성능이 낙관적으로 편향된다. 변환기는 train에서 fit하고 나머지 split에는 transform만 적용한다.

hold-out과 K-fold

hold-out은 한 번 분할해 빠르지만 분할에 따라 변동이 크다. 전체보다 작은 train subset으로 모델을 학습하므로 전체 데이터로 다시 적합할 최종 모델의 오류를 과대평가할 수 있다.

$$CV(K) = \sum_{k=1}^K (n_k/n) MSE_k$$
$$MSE_k = (1/n_k) \sum_{i \in C_k} (y_i - \hat{y}_i)^2$$

- 각 fold는 정확히 한 번 validation 역할을 한다.
- 서로 다른 fold는 겹치지 않고, 모든 fold의 합집합은 전체 표본을 이룬다.
- fold 크기가 다르면 n_k/n 으로 가중해 관측치 비중을 맞춘다.
- $k=n$ 이면 LOOCV이며 학습 횟수가 많아 계산비용이 크다.
- 분류는 class 비율을 유지하는 stratification, 시계열은 시간 순서를 지키는 분할이 필요하다.

K-means와 계층 군집

K-means

1. K개 centroid 초기화
2. 각 점을 가장 가까운 centroid에 할당
3. 군집 평균으로 centroid 갱신
4. 할당이 안정될 때까지 반복

초기값에 따라 지역 최솟값이 달라질 수 있다.

응집형 계층 군집

1. 각 점을 개별 군집으로 시작
2. 모든 군집쌍의 거리 계산
3. 가장 가까운 두 군집 병합
4. 거리를 갱신하며 한 군집까지 반복

덴드로그램 절단 높이로 군집 수를 정한다.

$$C_1 \cup \dots \cup C_K = \{1, \dots, n\}, \quad C_k \cap C_{k'} = \emptyset \quad (k \neq k')$$

군집 색상·번호 자체에는 의미가 없다. Euclidean 거리 기반 방법은 scale에 민감하므로 StandardScaler 등으로 단위를 맞춘다. 거리, linkage, 초기화, K를 바꿔 안정성과 도메인 해석 가능성을 함께 확인한다.

범주 정수 인코딩: 순서가 없는 범주를 맑음=1, 눈=2, 비=3처럼 넣으면 존재하지 않는 크기와 거리 관계가 생긴다. 명목형 범주는 one-hot 등 모델과 목적에 맞는 표현을 선택한다.

회귀 · 신경망

선형모델의 추정과 불확실성, 확률모델의 유도, 신경망 표현과 gradient 기반 최적화.

단순·다중 선형회귀

$$y_i = \beta_0 + \beta_1 x_i + \varepsilon_i \quad \cdot \quad \hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x$$

- β_0 : $x=0$ 일 때 조건부 평균의 기준값. $x=0$ 이 관측 범위 밖이면 실질 해석이 제한된다.
- β_1 : x 가 1 증가할 때 평균 예측값의 변화.
- 잔차 $e_i = y_i - \hat{y}_i$, RSS는 잔차 제곱합이다.

$$\hat{\beta}_1 = \Sigma(x_i - \bar{x})(y_i - \bar{y}) / \Sigma(x_i - \bar{x})^2 = \text{Cov}(x, y) / \text{Var}(x)$$

$$\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$$

다중회귀는 $y = X\beta + \varepsilon$ 로 표현한다. 다른 feature를 고정했을 때 한 feature의 계수를 부분 효과로 해석하지만, 높은 상관·상호작용·비선형성·인과 혼동을 별도로 고려한다. 설명변수 사이의 강한 다중공선성은 계수 분산을 키워 크기·부호·개별 효과 해석을 불안정하게 만들 수 있다.

정규방정식과 최소제곱

$$\text{RSS}(\beta) = \|y - X\beta\|^2$$

$$\partial \text{RSS} / \partial \beta = -2X^T(y - X\beta) = 0$$

$$\hat{\beta} = (X^T X)^{-1} X^T y$$

닫힌형 해는 $X^T X$ 가 가역일 때의 표현이다. 실제 수치 계산은 역행렬을 직접 만드는 대신 QR·SVD·선형방정식 풀이를 사용해 안정성을 높인다. feature가 선형종속이면 식별이 어려워진다.

추정 불확실성

값	의미
Standard Error	반복 표본에서 계수 추정량이 변동하는 규모
t statistic	$(\hat{\beta}_j - \beta_{j,0}) / SE(\hat{\beta}_j)$
p-value	귀무가설 아래 관측 통계량 이상이 나올 확률. 효과 크기 자체가 아님
confidence interval	추정 불확실성을 범위로 표현
RSE	잔차의 전형적인 크기를 y와 같은 단위로 표현
R²	평균 기준 대비 설명된 변동의 상대 비율

$$RSE = \sqrt{RSS/(n-p-1)} \quad \cdot \quad t = (\hat{\beta}_j - \beta_{j,0}) / SE(\hat{\beta}_j)$$

작은 p-value는 데이터·모형 가정 아래 귀무가설과의 불일치 정도를 나타낸다. 실무 중요성, 인과성, 재현성 또는 예측 성능을 자동으로 보장하지 않는다.

최대우도와 log-likelihood

$$L(\theta) = \prod_{i=1}^n p(y_i | x_i; \theta) \quad \cdot \quad \ell(\theta) = \log L(\theta) = \sum \log p(y_i | x_i; \theta)$$

log는 양수에서 단조 증가하므로 likelihood의 최댓값 위치를 바꾸지 않는다. 곱을 합으로 바꾸어 미분이 단순해지고 작은 확률을 계속 곱할 때 생기는 underflow도 줄인다. 최대 log-likelihood는 최소 negative log-likelihood와 같다.

로지스틱 회귀

$$z = \beta_0 + x^T \beta \quad \cdot \quad \sigma(z) = 1 / (1 + e^{-z}) \quad \cdot \quad p(y=1|x) = \sigma(z)$$

$$\log(p/(1-p)) = \beta_0 + x^T \beta$$

계수 β_j 는 x_j 가 1 증가할 때 log-odds 변화, $\exp(\beta_j)$ 는 odds ratio

유한한 입력에서 sigmoid 출력은 열린구간 $(0,1)$ 의 확률이며 threshold를 적용해 class로 변환한다. threshold는 오류 비용에 맞춰 조정하며 0.5가 항상 최선은 아니다. 양끝 포화 구간에서는 미분값이 매우 작아 깊은 망에서 기울기 소실을 일으킬 수 있다.

$$\text{Binary Cross-Entropy} = -[y \log p + (1-y) \log(1-p)]$$

다중분류 softmax 출력에는 $-\sum_c y_c \log p_c$ 를 사용한다. one-hot label에서는 정답 class 항만 남아 $-\log p_{\text{true}}$ 가 된다.

신경망의 구성

$$z^{(\ell)} = h^{(\ell-1)}W^{(\ell)} + b^{(\ell)}$$
$$h^{(\ell)} = \varphi(z^{(\ell)})$$

요소	역할
Linear / affine layer	입력의 가중합과 bias
activation	비선형성 추가. 여러 Linear만 쌓으면 하나의 Linear와 동등
hidden layer	중간 표현 생성
output layer	회귀값, logit, class 확률 등 작업에 맞는 출력

$$\text{ReLU}(z) = \max(0,z) \cdot \text{LeakyReLU}(z) = z \ (z \geq 0), \alpha z \ (z < 0)$$

ReLU 네트워크는 입력 공간을 여러 선형 영역으로 나눈 piecewise linear 함수를 만든다. Leaky ReLU는 음수 구간에 작은 기울기 α 를 남겨 unit의 gradient가 완전히 0이 되는 현상을 완화한다. 깊은 구조는 앞 층의 영역을 뒤 층에서 조합하여 비슷한 파라미터 수로도 더 많은 영역을 효율적으로 표현할 수 있지만 최적화와 일반화를 자동으로 보장하지 않는다.

shape와 파라미터 수

입력 차원 `d_in`, 출력 차원 `d_out` 인 fully connected layer의 weight는 `(d_in, d_out)`, bias는 `(d_out,)`로 볼 수 있다. 파라미터 수는 다음과 같다.

$$\text{parameter count} = d_{in} \cdot d_{out} + d_{out}$$

batch 입력 `x:(B,d_in)`과 weight `w:(d_in,d_out)`의 곱은 `(B,d_out)`이다. 다층망 전체 파라미터 수는 각 층의 weight와 bias를 모두 더한다.

경사하강법과 batch

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L(\theta)$$

η : learning rate, ∇L : 손실이 가장 빠르게 증가하는 방향

방식	한 번의 gradient에 쓰는 데이터	특성
full-batch	전체 train	안정적이지만 큰 데이터에서 갱신 비용·메모리 큼
stochastic	한 표본	갱신이 매우 잦고 잡음 큼
mini-batch	일부 표본 묶음	벡터 연산 효율과 적당한 gradient 잡음의 균형

학습률이 너무 크면 최소값 주변을 건너뛰어 손실이 진동·발산할 수 있고, 너무 작으면 수렴이 느리다. batch 크기, 초기화, 정규화, optimizer 설정과 함께 관찰한다.

Hyperparameter 탐색: Grid Search는 미리 정한 후보 조합을 체계적으로 검사하고, Random Search는 탐색 분포에서 제한된 횟수만큼 조합을 추출한다. 선택에는 validation 또는 CV를 사용하고 test는 최종 평가에 남긴다.

계산 그래프와 역전파

- 순전파로 각 연산의 입력·출력과 최종 loss를 계산한다.
- loss의 미분값 1에서 시작한다.
- 그래프를 역순으로 따라가며 국소 미분을 연쇄법칙으로 곱한다.
- 여러 경로에서 같은 변수로 도달한 gradient는 합산한다.
- optimizer가 계산된 gradient로 파라미터를 갱신한다.

$$z=f(x), L=g(z) \Rightarrow dL/dx = (dL/dz)(dz/dx)$$

역전파는 gradient를 계산하는 알고리즘이고 gradient descent는 그 gradient를 이용해 파라미터를 이동하는 최적화 방법이다.

PyTorch 기본 구조

```
class Model(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(10, 8), nn.ReLU(), nn.Linear(8, 1)
        )

    def forward(self, x):
        return self.net(x)

for x, y in loader:
    optimizer.zero_grad()
    pred = model(x)
    loss = criterion(pred, y)
    loss.backward()
    optimizer.step()
```

- `nn.Module` 은 layer·parameter를 등록하고 train/eval 상태와 장치 이동을 관리한다.
- `forward` 는 입력 tensor에서 출력 tensor까지의 순전파를 정의한다.
- PyTorch gradient는 기본적으로 누적되므로 batch마다 `zero_grad` 가 필요하다.
- 검증·추론에는 `model.eval()` 과 gradient 비활성화를 사용한다.

NLP · Transformer

텍스트를 수치 표현으로 바꾸는 과정, 순차 모델의 정보 전달, attention과 Transformer 사전학습 구조.

토큰화와 vocabulary

단위	특성
word	해석이 쉽지만 형태 변화·신조어로 vocabulary와 OOV 문제가 커짐
character	OOV가 작지만 sequence가 길어지고 의미 단위가 잘게 분해됨
subword	빈번한 문자열은 유지하고 희귀어를 조각내어 vocabulary와 길이의 균형

tokenizer는 텍스트를 token sequence와 integer id로 변환한다. special token은 padding, 문장 경계, 분류 위치, masking 등 모델별 기능을 표현한다. padding 위치는 attention mask로 계산에서 제외한다.

one-hot과 embedding

one-hot

vocabulary 크기 V 의 벡터에서 한 위치만 1이다. 희소하고 모든 서로 다른 단어가 직교하므로 의미 유사성을 직접 담지 못한다.

embedding

각 token을 낮은 차원 d 의 밀집 벡터로 매핑한다. 학습 중 문맥·작업에 유용한 방향이 weight에 형성된다.

$$\text{Embedding matrix } E \in \mathbb{R}^{V \times d} \quad \cdot \quad \text{token id } i \rightarrow E[i]$$

$$\text{cosine}(u,v) = (u \cdot v) / (\|u\| \|v\|)$$

cosine similarity는 방향의 유사성을 측정한다. embedding 공간의 관계는 학습 데이터와 목적에 의존하며 사회적 편향도 포함할 수 있다.

Word2Vec

방법	예측 방향
CBOW	주변 단어들 → 중심 단어
Skip-gram	중심 단어 → 주변 단어들

분포 가설은 비슷한 문맥에 등장하는 단어가 비슷한 의미를 가질 가능성이 높다는 생각이다. Word2Vec은 window 안의 동시출현 패턴으로 embedding을 학습한다. 계산량을 줄이기 위해 negative sampling처럼 일부 오답 단어만 사용하는 근사가 활용된다. 정적 embedding은 같은 단어가 문맥에 관계없이 하나의 벡터를 갖는 한계가 있다.

N-gram 언어모델

N-gram은 다음 단어 확률을 직전 N-1 개 단어의 빈도로 근사한다. 구조가 단순하지만 고정된 짧은 문맥만 사용하며, N이 커질수록 가능한 조합이 급증해 보지 못한 문맥의 희소성 문제가 커진다. smoothing은 0 확률을 완화하지만 긴 거리 의존성을 직접 해결하지는 못한다.

RNN과 BPTT

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$
$$\hat{y}_t = g(W_{hy}h_t + b_y)$$

- 같은 파라미터를 모든 시점에 공유한다.
- 이전 hidden state가 과거 정보를 현재로 전달한다.
- 순차 의존성 때문에 시점 방향 병렬화가 제한된다.
- BPTT는 시간축으로 펼친 계산 그래프에서 역전파한다.

vanishing / exploding gradient: 여러 시점의 Jacobian이 반복 곱해지며 크기가 0에 가까워지거나 폭증할 수 있다. 긴 의존성 학습을 어렵게 하며 gate 구조, 적절한 초기화, gradient clipping 등이 완화에 쓰인다.

LSTM과 GRU

구조	핵심 상태·gate	역할
LSTM	cell state, input-forget-output gate	정보 쓰기·삭제·출력 비율을 분리
GRU	hidden state, update-reset gate	별도 cell 없이 더 단순한 상태 조절

gate는 sigmoid 값으로 정보 흐름을 0과 1 사이에서 조절한다. LSTM·GRU도 무제한 문맥을 보장하지 않으며 sequence 길이와 학습 설정의 영향을 받는다.

Seq2Seq와 teacher forcing

encoder는 입력 sequence를 표현으로 바꾸고 decoder는 시작 token에서 출발해 출력 token을 순차 생성한다. 기본 구조가 마지막 encoder state 하나에 모든 입력을 압축하면 긴 문장에서 병목이 생긴다.

Teacher forcing

학습 시 decoder의 다음 입력으로 이전 정답 token을 사용한다. 학습이 안정적이지만 추론 시 자신의 예측을 입력으로 쓰는 차이 때문에 exposure bias가 생긴다.

Attention

decoder 시점마다 encoder의 전체 state를 점수화하고 가중합하여 현재 생성에 필요한 context를 만든다.

Scaled dot-product attention

$$\text{Attention}(Q,K,V) = \text{softmax}(QK^T/\sqrt{d_k})V$$

기호	기능
Query	현재 위치가 찾는 정보의 기준
Key	각 위치가 어떤 정보를 갖는지 비교할 표지
Value	attention weight로 실제 결합되는 내용
$\sqrt{d_k}$	차원이 커질 때 dot product가 커져 softmax가 포화하는 현상 완화

self-attention에서는 같은 입력 X의 서로 다른 선형 변환으로 Q, K, V를 만든다. cross-attention에서는 decoder query가 encoder의 key-value를 참조한다.

Multi-head와 위치 정보

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$
$$\text{MultiHead} = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

여러 head는 서로 다른 표현 부분공간과 관계를 병렬로 포착한다. self-attention만으로는 순서가 명시되지 않으므로 sinusoidal 또는 학습된 positional encoding을 token embedding에 더한다.

Transformer 블록

1. token embedding + position 정보
2. multi-head attention
3. residual connection + layer normalization
4. 위치별 feed-forward network
5. residual connection + layer normalization

residual connection은 sublayer 입력을 출력에 더하는 우회 경로를 만들어 정보와 gradient 흐름을 돕는다. 모든 위치가 직접 연결되어 장거리 정보 경로가 짧고 sequence 방향 병렬 학습이 가능하다. 표준 self-attention의 score 행렬은 길이 N에 대해 메모리·계산이 일반적으로 $O(N^2)$ 로 증가한다.

Encoder · Decoder · Mask

구조	attention 범위	대표 용도
encoder-only	양방향 self-attention	문장·token 이해, 분류, 추출형 QA
decoder-only	causal self-attention	다음 token 예측, 자기회귀 생성
encoder-decoder	encoder 양방향 + decoder causal + cross-attention	번역·요약·text-to-text 변환

causal mask는 위치 t가 미래 정답 token을 보지 못하게 attention logit을 차단한다. padding mask는 실제 문장이 아닌 padding 위치가 attention에 기여하지 못하게 한다.

BERT와 T5 사전학습

BERT · MLM 전체 token 중 일부를 예측 대상으로 선택한다. 선택된 token의 80%는 [MASK], 10%는 무작위 token, 10%는 원본을 유지한다. 양방향 encoder 표현을 학습한다.	T5 · Span corruption 연속된 span을 서로 다른 sentinel token으로 대체한다. decoder target은 sentinel과 삭제된 원래 span을 순서대로 복원한다. 모든 작업을 text-to-text 형식으로 통일한다.
--	--

사전학습 표현은 분류, 자연어 추론, 질의응답, 요약, 번역 등 여러 하위 작업으로 전이된다. downstream 성능은 사전학습 목적뿐 아니라 데이터, 모델 크기, fine-tuning 설정에 의존한다.

LLM · 평가 · 안전

Foundation Model의 학습·적응·생성 제어, 여러 층위의 평가와 실제 시스템의 신뢰성 경계.

Foundation Model

광범위한 데이터와 큰 계산량으로 일반 표현을 사전학습한 뒤 prompting, in-context learning, fine-tuning 등으로 다양한 작업에 적용하는 기반 모델이다. 하나의 고정 작업에만 맞춘 모델과 달리 재사용 범위가 넓지만 편향·오류도 여러 하위 시스템으로 전파될 수 있다.

$$p(x_1, \dots, x_T) = \prod_{t=1}^T p(x_t | x_{<t})$$

decoder-only 자기회귀 언어모델은 앞선 token을 조건으로 다음 token의 확률을 높이도록 학습한다. causal mask가 미래 token 접근을 차단한다.

Scaling과 능력 변화

경험적 scaling law에서는 모델 파라미터, 데이터, 계산량이 증가할수록 validation loss가 일정 범위에서 예측 가능한 power-law 패턴으로 감소한다. 자원의 한 축만 늘리면 비효율적일 수 있어 모델 크기와 token 수의 균형이 중요하다.

규모는 충분조건이 아니다. 데이터 품질, 중복·오염, 추론 비용, 사실성, 편향, 안전이 자동으로 해결되지 않는다.

일부 능력은 특정 규모 이후 급격히 생긴 것처럼 관측되는 emergent behavior를 보인다. 연속적인 확률 변화가 exact match 같은 이산 지표에서 급변해 보일 수도 있으므로 지표, 표본 수, 재현성을 함께 확인한다.

적응 방식

방식	weight 갱신	주요 목적
zero-shot prompting	없음	지시만으로 작업 수행
few-shot / ICL	없음	context 안의 예시로 형식·패턴 유도
SFT	있음	지시-응답 쌍으로 원하는 응답 행동 학습
parameter-efficient tuning	일부 추가 파라미터	전체 weight 갱신 비용 절감
RAG	생성 모델에는 보통 없음	질의 시 외부 지식을 context로 공급

fine-tuning은 표현 방식·업무 행동을 안정화하는 데 유용하지만 자주 바뀌는 사실을 갱신하는 지식 저장소와는 다르다. RAG는 최신·사내 지식을 검색해 제공하지만 검색 실패와 잘못된 문서가 생성 품질에 직접 영향을 준다.

SFT와 선호 정렬

1. **Pretraining**: 대규모 corpus에서 언어·지식 패턴을 학습

2. **SFT**: 지시와 모범 응답에 대한 supervised token loss 최소화
3. **Preference data**: 같은 지시에 대한 여러 답변의 선호 비교 수집
4. **Reward Model**: 선호 순서를 설명하는 scalar reward 학습
5. **Policy optimization**: reward를 높이되 기존 policy에서 과도하게 벗어나지 않도록 최적화

$$\text{objective} \approx E[r_{RM}(x,y)] - \beta \cdot \text{KL}(\pi_{\theta}(\cdot|x) \parallel \pi_{\text{ref}}(\cdot|x))$$

reward model은 사람 선호의 근사이며 진실·안전의 완전한 판정기가 아니다. policy가 reward의 허점을 이용하는 reward hacking이 가능하다. KL 패널티는 기존 모델과의 과도한 분포 이탈과 언어 품질 붕괴를 줄인다.

Decoding

방법	동작	특성
greedy	매 단계 최고 확률 token 선택	빠르고 결정적이지만 전역 시퀀스 최적은 아님
beam search	누적 점수가 높은 B개 가설 유지	탐색 범위 증가, 생성이 보수적·반복적일 수 있음
temperature	logit을 T로 나눠 분포 sharpness 조절	낮으면 일관성, 높으면 다양성 증가
top-k	상위 k개 token만 남겨 sampling	후보 수 고정
top-p	누적 확률 p를 넘는 최소 후보 집합에서 sampling	분포에 따라 후보 수 변화

$$p_T(x_i) = \exp(z_i/T) / \sum_j \exp(z_j/T)$$

낮은 temperature는 정확성을 보장하지 않으며 잘못된 고확률 응답도 더 일관되게 만들 수 있다. decoding 설정은 작업의 다양성·재현성·위험 허용도에 맞춘다.

Prompt와 context

- **system message**: 역할, 기본 행동, 제약을 전달하지만 준수를 완전히 보장하지 않는다.
- **user message**: 현재 요청과 입력 데이터.
- **assistant history**: 이전 응답이 후속 문맥을 형성한다.
- **few-shot example**: 원하는 입출력 형식을 context에서 보여 준다.
- **context window**: 한 호출에서 처리 가능한 token의 범위. 길다고 모든 정보를 동일하게 활용하는 것은 아니다.

명확한 작업, 출력 형식, 제약, 입력 경계를 분리하면 해석 모호성이 줄어든다. prompt injection은 외부 문서나 사용자 입력이 상위 지시를 무시하도록 유도하는 문제이며, 지시와 데이터의 구분·도구 권한 제한·출력 검증이 필요하다.

RAG와 Agent

구성	핵심 기능	주요 실패
retrieval	질의와 관련된 문서 후보 검색	관련 문서 누락, 권한 밖 문서 노출
reranking	후보의 관련성 재정렬	질문과 실제 답 근거의 불일치
generation	검색 context를 사용해 응답 작성	근거 무시, 과도한 추론, 환각
agent loop	모델이 도구 선택·호출·관찰·다음 행동 결정	잘못된 도구, 반복 loop, 권한 확대, 외부 부작용

Agent에는 허용 도구 목록, 인자 schema, 읽기·쓰기 권한 분리, 최대 반복 수, 비용 상한, 중요 작업의 사람 승인과 audit log가 필요하다.

언어모델 평가

평가 설계는 **대표성 있는 평가 데이터**, **목적에 맞는 지표**, **재현 가능한 프로토콜**을 함께 정의한다. prompt, decoding, 채점 기준, 모델 버전이 달라지면 같은 지표도 직접 비교하기 어렵다.

$$NLL = -(1/T)\sum_{t=1}^T \log p(x_t|x_{<t}) \cdot \text{Perplexity} = \exp(NLL)$$

perplexity가 낮으면 정답 token sequence에 더 높은 확률을 부여했다는 뜻이다. 사실성·유용성·안전·편향을 직접 평가하지는 않는다.

층위	대표 지표·방법	제한
token 확률	NLL, perplexity	생성 품질 전체를 설명하지 못함
정답형 task	accuracy, exact match, F1	허용 가능한 표현 다양성을 놓칠 수 있음
번역	BLEU	n-gram precision 중심, 의미 동일 표현에 민감
요약	ROUGE	겹침 recall 중심, 사실 불일치를 놓칠 수 있음
의미 유사성	embedding cosine, BERTScore	미세한 사실 오류·안전을 직접 판정하지 못함
사람 평가	정확성·유용성·명료성 rubric	비용, 평가자 불일치, 순서 효과
LLM judge	pairwise 비교·점수화	position·length·self bias

LLM judge는 답변 순서를 바꿔 재평가하고, rubric을 명시하며, 복수 judge와 사람 표본 검증을 함께 사용한다. 긴 답을 선호하는 length bias와 자신과 유사한 답을 선호하는 self-bias를 점검한다.

Benchmark 신뢰성

- **contamination**: 평가 문항 또는 매우 유사한 데이터가 학습에 포함되면 기억이 일반화처럼 보인다.
- **saturation**: 점수가 상한에 가까우면 모델 간 차이를 구분하기 어렵다.
- **distribution shift**: benchmark 분포와 실제 사용 분포가 다르면 순위가 유지되지 않을 수 있다.
- **prompt sensitivity**: 지시 형식·예시·답 순서에 따라 점수가 변할 수 있다.

- **aggregate masking**: 평균 점수는 특정 집단·위험 영역의 낮은 성능을 숨길 수 있다.

비공개 test, 시간 기준 분할, 중복 탐지, 여러 task와 실제 사용 시나리오, confidence interval과 오류 유형 분석을 함께 사용한다.

환각·편향·안전

환각 사실과 다르거나 제공된 근거로 뒷받침되지 않는 내용을 유창하게 생성하는 현상. 모델의 불확실성과 표현 확신은 일치하지 않는다.	편향 학습 데이터·annotation·평가지표·배포 환경의 불균형이 집단별 성능과 표현 차이로 나타날 수 있다.
유해성 공격적·위험한 내용, 개인정보 노출, 보안 취약 행동, 차별적 결과 등 사용 맥락의 피해 가능성.	과신·자동화 편향 사용자가 유창한 응답을 검증 없이 신뢰하거나 시스템이 사람 판단을 대체하면서 오류 영향이 확대되는 문제.

Jailbreaking은 역할극, 인코딩, 다단계 지시 등으로 모델의 안전 제약을 우회하도록 유도하는 공격이다. 단일 system prompt만으로 완전히 차단할 수 없으며 입력과 지시의 분리, 최소 도구 권한, 출력 검증, 공격 테스트와 운영 모니터링을 겹쳐 적용한다.

1. 권한이 통제된 신뢰 가능한 지식 검색
2. 응답 근거와 불확실성 표현
3. 구조화된 출력 schema와 후처리 validation
4. 고위험 판단의 사람 검토
5. 사실성·편향·공격·거부율을 포함한 지속 평가
6. 운영 log, incident 대응, model-prompt-index 버전 관리

CNN · 이미지 모델

공간 구조를 보존하는 합성곱의 계산 원리와 대표 이미지 모델의 설계 선택을 한 흐름으로 정리한다.

합성곱의 구조

국소 연결

작은 커널은 한 번에 인접한 픽셀만 본다. 같은 커널을 모든 위치에 적용하는 가중치 공유로 완전연결층보다 적은 파라미터로 위치별 패턴을 찾는다.

채널과 필터

하나의 출력 채널은 모든 입력 채널을 관통하는 필터 하나가 만든다. 입력이 $N \times C_{in} \times H \times W$, 커널이 $C_{out} \times C_{in} \times K_h \times K_w$ 이면 출력 채널은 C_{out} 이다.

계층적 특징

앞쪽은 모서리·질감 같은 국소 특징, 뒤쪽은 물체 부분과 전체 형태처럼 더 추상적인 특징을 표현한다. 층을 거칠수록 한 출력이 보는 수용영역도 커진다.

비선형성

선형 합성곱만 연속하면 전체도 하나의 선형변환이다. ReLU 같은 활성화가 층 사이에 들어가야 복잡한 결정 경계를 표현할 수 있다.

$$H' = \text{floor}((H - K_h + 2P_h) / S_h) + 1$$
$$W' = \text{floor}((W - K_w + 2P_w) / S_w) + 1$$

$$\text{합성곱 파라미터} = C_{out} \times C_{in} \times K_h \times K_w + C_{out}$$

출력 채널마다 bias 하나를 쓰는 경우

Pooling과 계산 자원

Max pooling 또는 stride convolution은 공간 크기를 줄여 계산량과 작은 위치 변화에 대한 민감도를 낮춘다. 대신 세밀한 위치 정보가 손실될 수 있으므로 pooling이 중요한 정보를 항상 보존한다고 가정하면 안 된다.

측정값	주로 말해 주는 것	주의점
파라미터 수	학습할 가중치와 모델 저장 규모	적어도 큰 feature map에서는 계산이 많을 수 있다.
Activation memory	중간 feature map 저장량	학습에서는 역전파를 위해 여러 activation을 보관한다.
FLOPs	한 번의 forward에 필요한 산술 연산량	실제 속도는 메모리 접근과 하드웨어에도 좌우된다.

대표 모델의 설계 변화

모델	핵심 설계	해석
AlexNet	5개 convolution + 3개 FC, ReLU, 큰 첫 커널과 stride	대규모 이미지 분류에서 깊은 CNN의 가능성을 보여 준 초기 구조
VGG	3×3 convolution 반복, 블록 사이 2×2 max pooling	규칙적이고 단순하지만 FC 파라미터와 convolution 계산량이 크다.
ResNet	<code>F(x)+x</code> identity shortcut, bottleneck block	깊은 plain network의 degradation과 최적화 문제를 완화한다.
MobileNet	depthwise spatial convolution + 1×1 pointwise convolution	공간 처리와 채널 혼합을 분리해 모바일 환경의 연산량을 줄인다.

작은 커널을 쌓는 이유. stride 1의 3×3 convolution 두 층은 5×5 수용영역을 만들면서, 채널 수가 같을 때 약 $18C^2$ 대 $25C^2$ 로 파라미터가 적고 활성화를 한 번 더 적용할 수 있다.

Depthwise separable / standard 비용 비 $\approx 1/C + 1/K^2$

$K \times K$ 커널, 입력·출력 채널을 C 로 단순화한 경우

ViT · 학습 전략

이미지를 token으로 보는 Transformer의 구조와, 모델 성능을 안정적으로 끌어내기 위한 실전 선택을 구분한다.

이미지 Attention과 ViT

ViT는 이미지를 고정 크기 patch로 나누고 각 patch를 token embedding으로 바꾼다. self-attention은 모든 patch 사이의 관계를 직접 계산하므로 전역 맥락을 빠르게 연결하지만, CNN의 국소성 같은 inductive bias가 약해 충분한 데이터와 정규화가 중요하다.

Query
현재 token이 어떤 정보를 찾는지 나타낸다.

Key
Query와 비교해 관련성을 계산하는 기준이다.

Value
attention weight로 가중합하여 실제 전달하는 내용이다.

Attention(Q,K,V) = softmax(QK^T / √d_k)V

위치 표현	특징	주요 고려점
학습형 absolute	데이터에 맞춘 위치 vector를 학습	해상도와 patch 격자가 달라지면 보간이 필요할 수 있다.
고정 sinusoidal	학습하지 않는 규칙적 위치 신호	길이 변화에는 유연하지만 과제 맞춤 적응은 제한된다.
relative	token 사이 거리와 방향을 표현	입력 해상도 변화에 더 유연할 수 있다.

DeiT는 CNN teacher의 출력 분포와 실제 label을 함께 사용해 ViT student를 distillation한다. Swin Transformer는 window attention으로 계산을 줄이고 shifted window로 창 사이 정보를 교환하며 계층적 feature map을 만든다.

활성화와 초기화

함수	장점	한계
Sigmoid	0~1 출력, 확률 해석	포화 기울기 소실, 양수 중심, exp 비용
tanh	-1~1, 0 중심	포화 구간의 기울기 소실
ReLU	양수 구간 비포화, 계산이 단순	음수 구간 gradient가 0인 dying ReLU
Leaky ReLU	음수 구간에 작은 gradient 유지	음수 기울기 크기를 정해야 하며 항상 더 좋은 것은 아니다.
ELU	음수 출력과 부드러운 변화	exp 연산과 α 선택이 필요하다.

모든 가중치를 0으로 두면 같은 층의 뉴런들이 같은 gradient를 받아 대칭이 깨지지 않는다. 무작위 초기화는 대칭을 깨뜨려 층을 지나며 activation과 gradient의 분산이 지나치게 커지거나 작아지지 않도록 규모를 맞춰야 한다.

Xavier

입력·출력 차원을 고려하며 tanh 계열에서 signal scale을 유지하는 데 주로 사용한다.

He

ReLU의 음수 절반이 0이 되는 성질을 고려해 더 큰 분산을 사용한다.

정규화와 학습률

전략	핵심 동작	확인할 점
L1	절댓값 합을 벌점으로 추가	일부 가중치를 정확히 0으로 만들어 희소성을 유도할 수 있다.
L2 / weight decay	큰 가중치를 부드럽게 감소	규제가 너무 강하면 underfitting이 생긴다.
Dropout	훈련 중 unit을 확률적으로 제거	추론 모드 전환과 scaling을 정확히 적용한다.
Data augmentation	label을 보존하는 변환으로 학습 sample 확장	crop-flip·색 변화가 과제의 정답 의미를 바꾸지 않아야 한다.
Early stopping	validation 성능이 나빠지기 전 checkpoint 선택	test set은 선택에 쓰지 않는다.

학습률이 너무 크면 loss가 발산하거나 진동하고, 너무 작으면 수렴이 지나치게 느리다. step decay는 지정 epoch에서 계단식으로, cosine schedule은 곡선을 따라 매끄럽게 학습률을 줄인다. 곡선이 멈춘 것처럼 보이면 초기화, 학습률, 모델 용량, 정규화 강도를 함께 점검한다.

전이학습

- 사전학습 모델과 동일한 입력 크기·채널 순서·정규화 통계를 적용한다.
- Linear probing**: backbone을 freeze하고 새 분류 head만 학습해 전이 가능한 특징인지 확인한다.
- Fine-tuning**: 낮은 학습률로 일부 또는 전체 backbone을 새 과제에 맞게 조정한다.
- label을 보존하는 augmentation과 learning-rate scheduler를 적용한다.
- validation으로 설정을 선택하고 test는 최종 평가에만 사용한다.

핵심 수식

주제	수식
평균 학습 목적	$\hat{f} = \arg \min_{f \in F} (1/n) \sum L(y_i, f(x_i))$
MSE · RMSE	$MSE = (1/n) \sum (y - \hat{y})^2, RMSE = \sqrt{MSE}$
R²	$1 - RSS/TSS$
Precision · Recall	$TP/(TP+FP), TP/(TP+FN)$
Specificity · NPV	$TN/(TN+FP), TN/(TN+FN)$
F1	$2PR/(P+R)$
K-fold	$\sum (n_k/n) \cdot Error_k$
선형회귀	$\hat{y} = \hat{\beta}_0 + x^T \hat{\beta}$
정규방정식	$\hat{\beta} = (X^T X)^{-1} X^T y$
t 통계량	$(\hat{\beta}_j - \beta_{j,0}) / SE(\hat{\beta}_j)$
sigmoid	$\sigma(z) = 1 / (1 + e^{-z})$
binary CE	$-[y \log p + (1-y) \log (1-p)]$
gradient update	$\theta \leftarrow \theta - \eta \nabla L(\theta)$
ReLU	$\max(0, z)$
cosine similarity	$(u \cdot v) / (\ u\ \ v\)$
attention	$\text{softmax}(QK^T / \sqrt{d_k})V$
언어모델 분해	$\prod p(x_t x_{<t})$
perplexity	$\exp(\text{평균 negative log-likelihood})$

혼동하기 쉬운 구분

A	B	구분점
<code>append</code>	<code>extend</code>	객체 하나 추가 vs iterable 원소를 각각 추가
<code>sort</code>	<code>sorted</code>	원본 변경·None 반환 vs 새 정렬값 반환
<code>d[key]</code>	<code>d.get</code>	키 누락 시 KeyError vs 기본값 반환 가능
<code>response.text</code>	<code>response.json()</code>	본문 문자열 vs JSON을 Python 객체로 변환
list 연산	ndarray 연산	연결·반복 가능 vs 원소별 vectorization
<code>loc</code>	<code>iloc</code>	label·조건 vs 정수 위치
<code>count</code>	<code>size</code>	NaN 제외 유효값 수 vs NaN 포함 행 수
<code>cut</code>	<code>qcut</code>	값 범위 기준 vs 분위수·비슷한 관측치 수
loss	metric	학습 최적화 목적 vs 성능 해석 지표
validation	test	모델 선택 vs 최종 1회 평가
Precision	Recall	양성 예측의 신뢰도 vs 실제 양성 탐지율
K-means	계층 군집	K와 centroid 반복 vs 병합 이력과 덴드로그램
선형회귀	로지스틱 회귀	연속 평균·제곱오차 vs class 확률·cross-entropy
역전파	경사하강법	gradient 계산 vs gradient로 parameter 갱신
CBOW	Skip-gram	주변→중심 vs 중심→주변
encoder-only	decoder-only	양방향 이해 vs causal 생성
SFT	RLHF	모범 응답 지도학습 vs 비교 선호 reward 최적화
top-k	top-p	후보 개수 고정 vs 누적 확률 기준
fine-tuning	RAG	weight·행동 변경 vs 추론 시 지식 공급
perplexity	사실성 평가	token 확률 품질 vs 주장과 근거의 일치